

Supply Chain Tracker database and function report

Project reviewed from Database-Supply-Chain-Tracker-main.zip

Application type	Supply chain and inventory tracking system
Backend	FastAPI, SQLAlchemy, Pydantic, JWT authentication, bcrypt password hashing
Frontend	Vue, Vite, reusable resource panels, dashboard tabs
Database	PostgreSQL-style SQL schema, constraints, indexes, triggers, stored procedures
Main focus reviewed	Inventory integrity, order fulfilment, shipment tracking, audit logs

1. Executive summary

The project is a supply chain tracker with authenticated users, catalog management, partners, warehouses, inventory, purchase orders, customer orders, shipments, inventory logs, and status history. The database design is normalized around products, stock locations, orders, purchase orders, and shipment movements.

The most important implementation area is stock movement control. This is the part I would optimize and protect first because a supply chain system is only useful if inventory quantities, shipment status, and stock audit logs stay accurate.

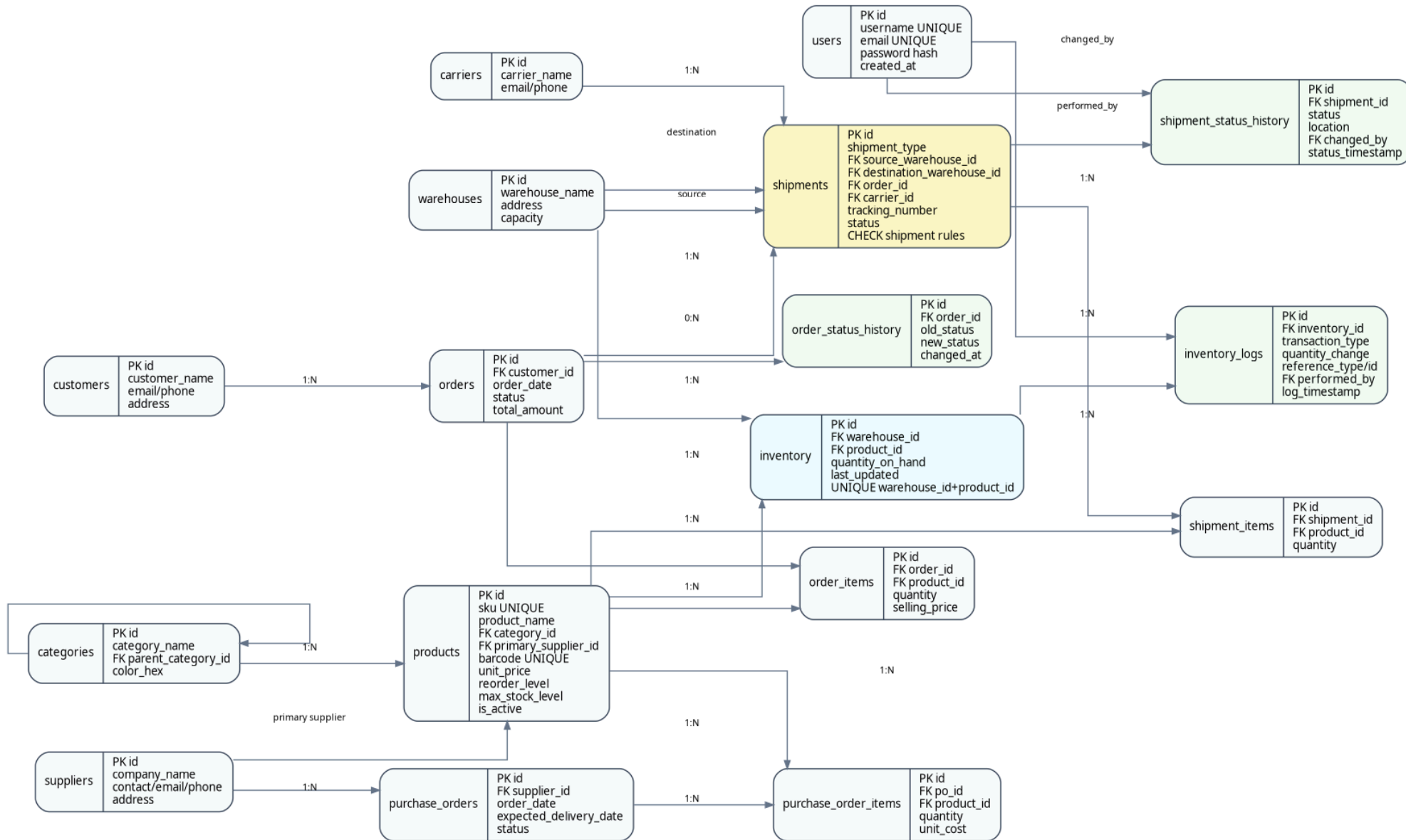
- Security is implemented through password hashing, JWT access tokens, and route protection.
- Data integrity is implemented through foreign keys, unique constraints, shipment CHECK constraints, status history, inventory logs, triggers, and stored procedures.
- Performance is improved with indexes on foreign keys, status fields, and inventory log references.
- A few consistency improvements are still recommended before production use.

2. Source files reviewed

Area	Files
Database schema	sql/schema.sql, backend/models.py
Migrations	sql/migrations/0001_catalog_enrichment.sql, 0002_catalog_trim.sql, 0003_link_orders_shipments.sql
Indexes	sql/indexes.sql
Triggers	sql/triggers.sql
Stored procedures	sql/procedures.sql
Authentication	backend/auth.py, backend/routes/auth.py
API routes	backend/routes/catalog.py, partners.py, warehouses.py, purchase_orders.py, orders.py, shipments.py, inventory_logs.py
Frontend screens	frontend/src/views/LoginView.vue, RegisterView.vue, DashboardView.vue, frontend/src/components/*.vue

3. Database design

The ER diagram below shows the main tables and relationships. The central tables are products, inventory, orders, purchase orders, shipments, and audit-history tables.



ER diagram generated from the SQLAlchemy models and SQL schema files.

Supply Chain Tracker report

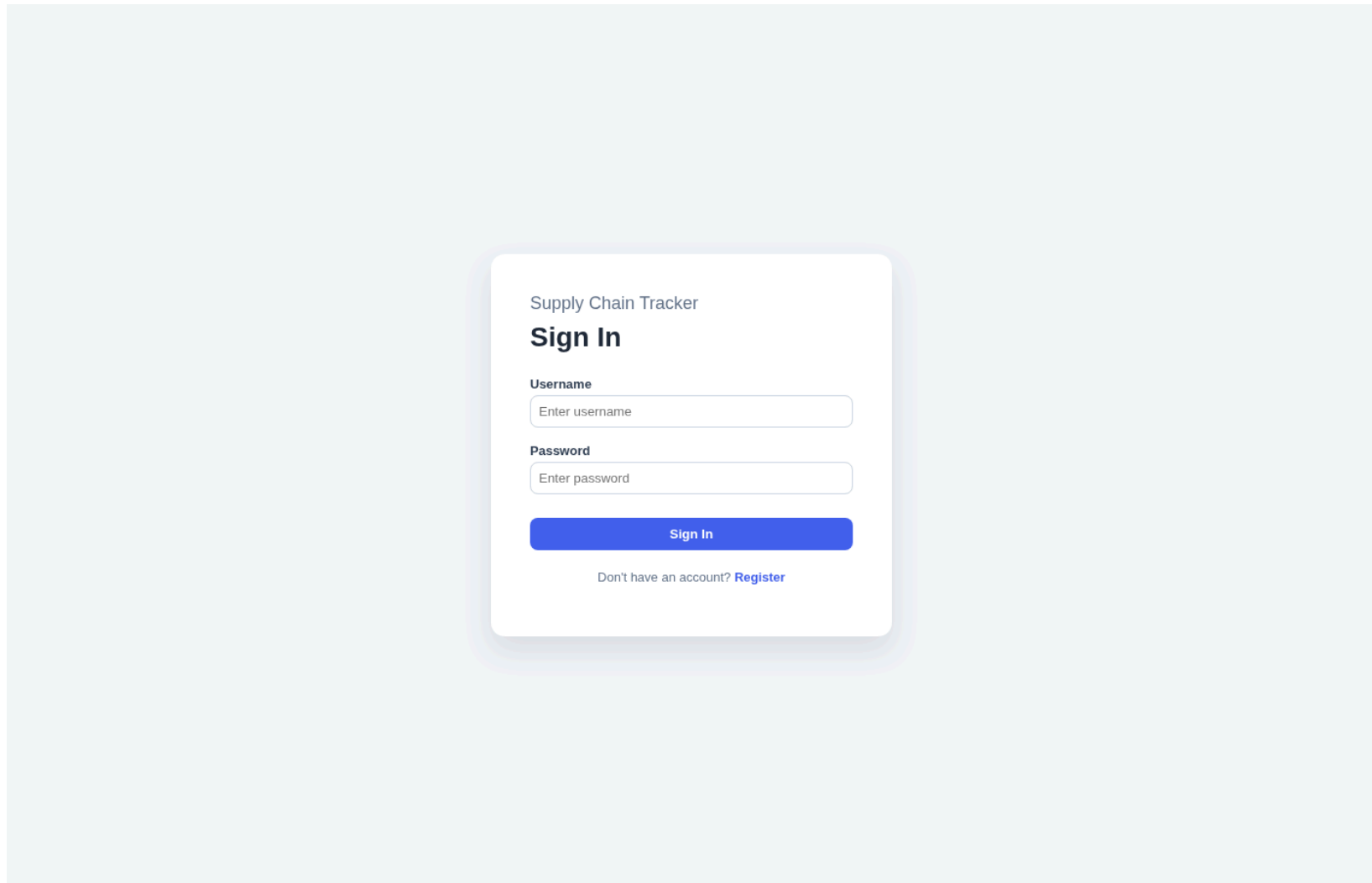
4. Main tables and purpose

Table	Purpose	Important keys or controls
users	Application users.	Unique username and email. Password stored as hash.
categories	Product category tree.	Self reference through parent <code>category_id</code> .
suppliers, customers, carriers	Business partners used by purchase orders, customer orders, and shipments.	Primary keys referenced by operational tables.
products	Product master data.	Unique SKU, optional unique barcode, category, supplier, price, reorder level.
warehouses	Stock locations.	Referenced by inventory and shipments.
inventory	Quantity per product and warehouse.	Unique <code>warehouse_id</code> plus <code>product_id</code> . Negative inventory blocked by trigger.
<code>purchase_orders</code> and <code>purchase_order_items</code>	Inbound supplier purchase flow.	PO items link products and costs.
<code>orders</code> and <code>order_items</code>	Customer demand and sales amount.	Items are used to generate customer delivery shipments.
<code>shipments</code> and <code>shipment_items</code>	Transfer and customer delivery flows.	CHECK constraints enforce correct target fields by <code>shipment_type</code> .
<code>inventory_logs</code>	Audit ledger for stock changes.	References inventory row, transaction type, quantity delta, user, and reference document.
<code>shipment_status_history</code>	Shipment status audit trail.	Tracks status, location, user, timestamp, and notes.
<code>order_status_history</code>	Order status audit trail created by trigger.	Tracks old and new status on order update.

5. Application screens

The screens below are reconstructed from the implemented Vue components and labels, using sample data. They show how the implemented functions are exposed to the user.

5.1 Login screen

The image shows a login screen for a web application titled "Supply Chain Tracker". The background is a solid light blue. In the center, there is a white rounded rectangle with a subtle drop shadow. Inside this rectangle, the text "Supply Chain Tracker" is at the top in a small, dark font. Below it, the heading "Sign In" is displayed in a larger, bold, dark font. There are two input fields: the first is labeled "Username" and contains the placeholder text "Enter username"; the second is labeled "Password" and contains the placeholder text "Enter password". Below these fields is a solid blue button with the text "Sign In" in white. At the bottom of the white rectangle, there is a link that says "Don't have an account? Register", where "Register" is a blue hyperlink.

Login screen. The login form calls POST /auth/login and stores the JWT token in localStorage.

5.2 Dashboard catalog screen

SC

Supply Chain Tracker

Catalog

Partners

Warehouses

Purchase Orders

Orders

Shipments

Settings

Home / Catalog

Jjp_admin

Categories

Name*

Electrical parts

Parent category

Select...

Color

#4361ee

Description

Components and cable assemblies

Save

Export

Reset

Total Record(s) Found: 3

Search...

Edit	Del	Name	Parent category	Color
		Electrical parts	—	#4361ee
		Packaging	—	#90be6d

Products

SKU*

SC-1001

Name*

Harness cable kit

Category

Electrical parts

Unit price*

18.50

Primary supplier

Thai Cable Supply

Reorder level*

25

Max stock

250

☒ Active

Total Record(s) Found: 2

Search...

Image	SKU	Name	Brand	Category	Unit price	Reorder level	Active
	SC-1001	Harness cable kit	Satys	Electrical parts	18.50	25	Yes

Dashboard screen. The sidebar exposes Catalog, Partners, Warehouses, Purchase Orders, Orders, Shipments, and Settings.

Supply Chain Tracker report

6. Related functions

Feature	Frontend component or screen	API endpoint/function	Result
Register	RegisterView.vue	POST /auth/register -> register()	Creates user with bcrypt hashed password.
Login	LoginView.vue	POST /auth/login -> login()	Returns JWT access token.
Change password	Dashboard settings	PUT /auth/change-password -> change_password()	Verifies current password and stores new hash.
Categories	ResourcePanel in DashboardView.vue	GET/POST/PUT/DELETE /categories	CRUD category master data. Prevents circular hierarchy on update.
Products	ResourcePanel in DashboardView.vue	GET/POST/PUT/DELETE /products	CRUD product master data. Validates SKU and barcode uniqueness.
Partners	ResourcePanel	Suppliers, customers, carriers endpoints	CRUD partner master data.
Warehouses	ResourcePanel and InventoryPanel	GET/POST/PUT/DELETE /warehouses and /inventory	Manages warehouses and stock rows.
Low stock	InventoryPanel	GET /inventory/low-stock	Returns rows where quantity is at or below product reorder level.
Purchase orders	PurchaseOrdersPanel.vue	POST /purchase-orders, /items, /receive	Creates POs, adds items, receives goods into inventory.
Customer orders	OrdersPanel.vue	POST /orders, /items, /ship	Creates orders, adds items, creates shipment through stored procedure.
Shipments	ShipmentsPanel.vue	POST /shipments, /ship, /deliver, /history	Tracks shipment lifecycle and status history.
Inventory logs	API only	GET /inventory-logs	Read-only ledger for stock movements.

7. Example result when a function is called

Example selected function: ship a customer order. This function is important because it validates the order, checks stock, creates a customer delivery shipment, copies order items to shipment items, logs the initial shipment status, and changes the order status to processing.

Example function call result

Order fulfilment creates a shipment after validating the order and stock.

REQUEST

POST /orders/1/ship

Authorization: Bearer <JWT token>

```
{
  "source_warehouse_id": 1,
  "carrier_id": 1,
  "tracking_number": "DHL-TH-2026-0001"
}
```

RESPONSE 201 CREATED

```
{
  "id": 12,
  "shipment_type": "CUSTOMER_DELIVERY",
  "source_warehouse_id": 1,
  "destination_warehouse_id": null,
  "order_id": 1,
  "carrier_id": 1,
  "tracking_number": "DHL-TH-2026-0001",
  "status": "pending",
  "source_warehouse": { "id": 1, "warehouse_name": "Main
warehouse" },
  "carrier": { "id": 1, "carrier_name": "DHL" },
  "items": [
    { "product_id": 1, "quantity": 5 }
  ]
}
```

Example API request and response for POST /orders/{order_id}/ship.

7.1 Function path

```
Frontend OrdersPanel.vue
  handleFulfill(order)
    -> shipOrder(order.id, payload)

frontend/src/api/orders.js
  POST /orders/{id}/ship

backend/routes/orders.py
  ship_order(...)
    -> SELECT sp_ship_order(:order_id, :warehouse_id, :carrier_id, :tracking)

sql/procedures.sql
  sp_ship_order(...)
    -> validate pending order
    -> validate order has items
    -> validate warehouse has enough stock
    -> create shipment
    -> copy order items into shipment_items
    -> insert shipment_status_history
    -> update order status to processing
    -> return shipment_id
```

8. Security and data integrity implementation

Implemented security and data-integrity controls

Security

- 1 Passwords are hashed with `bcrypt` before storage.
- 2 Login returns a JWT access token with expiry.
- 3 Protected routes require `HTTPBearer` token validation through `get_current_user`.

Data integrity

- 1 Foreign keys connect products, orders, shipments, inventory and partners.
- 2 Shipment CHECK constraints block invalid transfer and customer-delivery records.
- 3 Triggers block negative inventory and protect non-pending orders/shipments from deletion.
- 4 Inventory logs and status history tables keep an audit trail.

8.1 Security controls

Control	Implementation	Reason
Password hashing	backend/auth.py uses bcrypt.hashpw and bcrypt.checkpw.	Passwords are not stored in plain text.
JWT token	create_access_token adds an expiry and signs the payload with HS256.	Users receive a time-limited access token after login.
Protected routes	Most routers use Depends(get_current_user).	Catalog, warehouse, inventory, PO, order, and shipment data require authentication.
CORS restriction	main.py allows http://localhost:5173.	Limits browser access during development.

8.2 Data integrity controls

```
-- Negative inventory protection in sql/triggers.sql
CREATE OR REPLACE FUNCTION trg_fn_prevent_negative_inventory() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.quantity_on_hand < 0 THEN
        RAISE EXCEPTION 'Inventory cannot go negative (warehouse %, product %)',
            NEW.warehouse_id, NEW.product_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_negative_inventory
BEFORE UPDATE ON inventory
FOR EACH ROW EXECUTE FUNCTION trg_fn_prevent_negative_inventory();
```

Control	Location	Why it matters
Unique SKU and barcode	products table and catalog route validation	Prevents duplicate product master data.
Unique inventory row	inventory UNIQUE (warehouse_id, product_id)	Prevents two stock balances for the same product in the same warehouse.
Shipment type CHECK	shipments table	A transfer must have a destination warehouse. A customer delivery must have an order. Invalid mixed records are blocked.
Negative inventory trigger	sql/triggers.sql	Protects stock even if a bug or direct SQL update bypasses the API.
Protected delete trigger	sql/triggers.sql	Blocks deletion of non-pending orders and shipments.

Inventory logs	inventory_utils.py and inventory_logs table	Creates traceability for stock movements and corrections.
Status history	shipment_status_history and order_status_history	Preserves operational history instead of overwriting only the latest status.
Stored procedure for order shipping	sql/procedures.sql and backend/routes/orders.py	Keeps validation and shipment creation inside one database transaction.

9. Why this part was optimized

I chose inventory movement and shipment fulfilment as the optimization target because this is the highest-risk business process in the system. If stock is wrong, the system can accept orders that cannot be shipped, create false availability, lose traceability, and damage operational planning.

- It is high volume. Stock and shipment queries happen more often than master data changes.
- It is high impact. A wrong quantity directly affects fulfilment decisions.
- It is shared by several functions. Purchase order receiving, shipment outbound, transfers, low-stock reports, and logs all use inventory.
- It needs auditability. Every manual correction and operational movement should create a traceable log.
- It benefits from database-level protection. API checks alone are not enough because future code or direct SQL scripts could bypass them.

10. Performance optimization implemented

The sql/indexes.sql file adds indexes on the fields most likely to be used in joins and filters. This supports faster list screens, status filters, and audit-log lookup.

Index group	Examples	Benefit
Foreign-key joins	product_id, category_id, supplier_id, customer_id, warehouse_id	Faster joins between master data and transactions.
Status filters	purchase_orders.status, orders.status, shipments.status	Faster dashboard filtering by operational status.
Log lookup	inventory_logs.inventory_id and inventory_logs(reference_type, reference_id)	Faster audit trail lookup for a PO, shipment, or stock row.
Shipment lookup	source_warehouse_id, destination_warehouse_id, order_id, carrier_id	Faster shipment tracking by source, destination, customer order, or carrier.

11. Recommended corrections before production

- Replace the fallback SECRET_KEY with a required environment variable. The current fallback value should not be usable in production.
- Add role-based access control if not every authenticated user should edit all master and transaction data.
- Align schema.sql and SQLAlchemy models. Example: the SQLAlchemy model includes shipment_status_history.changed_by, but schema.sql does not show this column.

- When delivering a transfer shipment, update destination inventory as well as shipment status. The current deliver route records history but does not add destination stock.
- When delivering any shipment, set shipment.status to delivered. The current deliver route inserts history but does not update the shipment status field.
- Avoid mutable default lists in Pydantic models. Use Field(default_factory=list) for item lists.
- Consider running purchase order receiving through the stored procedure sp_receive_purchase_order for consistency with sp_ship_order.

12. Conclusion

The project has a clear data model and a practical separation between frontend screens, API routes, database tables, and integrity controls. The best implemented part is the inventory and shipment control layer because it combines API checks, database constraints, triggers, stored procedures, and audit logs. This is the right area to optimize first in a supply chain tracker because it protects the most important data: stock accuracy and transaction traceability.